

Detecting and Preventing Input Injection in LLM-based Agents with 3-Layer Filtering

YING-DAR LIN¹ (Fellow, IEEE), MINH-BANG LE², REN-HUNG HWANG³ (Senior Member, IEEE), YUAN-CHENG LAI⁴, DIDIK SUDYANA⁵

¹Department of Computer Science, National Yang Ming Chiao Tung University, Taiwan

²EECS International Graduate Program, National Yang Ming Chiao Tung University, Taiwan

³College of Artificial Intelligence, National Yang Ming Chiao Tung University, Taiwan

⁴Department of Information Management, National Taiwan University of Science and Technology, Taiwan

⁵Computer and Network Center, National Cheng Kung University, Taiwan

ABSTRACT Large Language Models (LLMs) have evolved into autonomous agents that reason over user prompts, retrieved documents, and tool outputs within one accumulated context. Because these channels accumulate, an attacker can split instructions across them so that each fragment looks harmless alone and the malicious intent surfaces only when the agent combines them, a pattern we call a correlated multi-channel attack. However, current defenses inspect only individual inputs or actions and consequently have no visibility into the cross-channel history where such attacks form. Therefore, an effective defense must detect both known and unseen attacks from that history while keeping false positives and semantic-auditing cost bounded. This work proposes InteractionGuard, which audits the accumulated interaction history through 3-layer filtering: an unsupervised anomaly detector for unseen attacks, a supervised transformer classifier for known patterns, and an LLM-based semantic auditor for reasoning-level verification. On a LangGraph-based agent testbed, InteractionGuard achieves an F1-score of 0.9614 ± 0.0047 and an FPR of 0.0165 ± 0.0092 over five seeds, while reducing mean latency by 72.4% compared with standalone semantic auditing. In a separate attack evaluation, it reduces the mean attack success rate from 0.1574 with the combined channel-specific baselines to 0.0000.

INDEX TERMS LLM-based Agent, Prompt Injection, 3-Layer Filtering, Anomaly Detection, Transformer, Large Language Models

I. INTRODUCTION

LARGE Language Models (LLMs) have evolved from passive text generators into autonomous agents that reason, retrieve external knowledge, and invoke tools through frameworks such as ReAct [1], [2]. Because agents act on behalf of users, manipulating their reasoning can cause unauthorized tool use or data leakage [3]. A fundamental cause is Code/Data Confusion: an LLM cannot reliably distinguish trusted instructions from untrusted external content [4].

In standalone LLMs, the main threat is Direct Prompt Injection (DPI), where an attacker places malicious instructions directly in the user prompt [5]; such single-channel attacks can often be mitigated through prompt filtering [2]. An agent's context, however, accumulates information from user prompts, retrieved documents, and tool outputs [3], so adversarial instructions can be distributed across channels such that each fragment looks benign alone and the malicious

intent surfaces only after the agent combines them. We refer to this pattern as a correlated multi-channel attack. Similar risks arise from poisoned RAG documents [6] and malicious tool outputs [7]; since the malicious intent emerges only after cross-channel aggregation, defenses that inspect each input independently are fundamentally limited.

Defending against correlated multi-channel attacks requires methods that can assess how information from different channels combines inside the agent's reasoning context, yet existing defenses remain confined to narrower scopes. They can be grouped into input-level and execution-level approaches. Input-level defenses use supervised classifiers, such as ProtectAI [8] and Sentinel [5], to identify injection patterns; they depend on labeled attack data, struggle with unseen or obfuscated variants [9], and assume that malicious intent is visible within a single input [4]. Execution-level defenses, such as MindGuard [4], DRIFT [10], and

AgentSentry [11], validate runtime decisions; they analyze local actions in isolation, impose rule-based constraints that reduce agent flexibility [10], [12], and add overhead when LLM-based validation is repeated [13].

These limitations resolve into three gaps: input-level classifiers miss unseen variants; aggressive filtering without corroboration against known patterns risks many false positives; and no defense inspects the full accumulated history, so correlated attacks remain invisible until damage is done.

To address these gaps, this work shifts detection from isolated inputs to interaction-history auditing: the security of an agent is judged by whether its accumulated history remains aligned with the user’s original intent. We call the resulting framework InteractionGuard, a cascaded 3-layer filtering architecture in which each layer targets one gap. The first layer, an anomaly detector trained only on benign traces, needs no attack labels and flags histories that deviate from benign behavior, closing the unseen-attack gap. The second layer, a supervised transformer classifier, cross-checks the histories flagged by Layer 1 against known attack patterns and filters out benign anomalies, closing the false-positive gap. The third layer, an LLM-based semantic auditor, reasons over the full history to verify instruction–data separation and alignment with the authorized task, closing the interaction-level visibility gap.

The contributions of this work are fourfold. First, it formalizes correlated multi-channel input injection, whose fragments each pass single-channel inspection. Second, it proposes interaction-history auditing, which treats the accumulated history as the unit of inspection. Third, it instantiates this paradigm as a cascaded 3-layer filtering architecture. Fourth, it evaluates InteractionGuard against prompt injection, RAG poisoning, tool injection, and correlated attacks on a LangGraph-based agent testbed.

The evaluation is organized around three questions. First, does the full three-layer cascade offer a better security–latency trade-off than reduced-layer configurations? Second, how does InteractionGuard’s attack prevention compare with existing channel-specific defenses? Third, how effectively does the cascade mitigate the latency bottleneck of the Layer 3 auditor under realistic thresholds and workloads?

The rest of this paper is organized as follows. Section II reviews background and related work. Section III presents the problem formulation. Section IV describes InteractionGuard. Section V presents the implementation. Section VI reports the evaluation results. Section VII concludes the paper.

II. BACKGROUND AND RELATED WORK

This section introduces LLM-based agents, RAG, and input injection attacks, then reviews existing defenses.

A. LLM-BASED AGENTS

Modern LLM-based agents execute an iterative reasoning–action–observation loop, commonly the ReAct framework [1]. At each turn, the agent conditions its next action on

everything observed so far, including prior prompts, retrieved documents, tool outputs, and its own reasoning traces. Unlike a standalone LLM, an agent’s decision at turn i therefore depends on the entire accumulated history (Section III). The same accumulation lets Code/Data Confusion [4] compound over time: every channel is represented as undifferentiated text in one context window, so content introduced several turns earlier can still influence a later decision after any marker of its origin is lost.

B. RETRIEVAL-AUGMENTED GENERATION

Retrieval-Augmented Generation (RAG) grounds model responses in documents retrieved from a vector database through embedding similarity [6], [14]. It also introduces external memory as an attack surface: if an attacker poisons the knowledge base, the agent may later retrieve this content, treat it as trusted context, and let it influence reasoning, trigger tool calls, and persist across steps.

C. INPUT INJECTION

Direct prompt injection places malicious instructions in the user input [5]. Indirect attacks inject instructions through retrieved documents, emails, web pages, or tool outputs, where they appear as ordinary data. Correlated multi-channel attacks are the most severe class because they defeat any defense that inspects channels independently. They proceed in three phases: in the seeding phase, the attacker places benign-looking signals into one or more channels; in the activating phase, a legitimate user request causes the agent to retrieve or process them; in the correlating phase, the agent aggregates them and forms a coherent malicious behavior.

EchoLeak in Microsoft 365 Copilot maps onto this pattern [15]: a hidden instruction seeded in an email is triggered by a routine summarization request and combined with internal data to exfiltrate it through a renderable resource. No individual step is malicious; the attack is visible only once the three phases are read as one interaction history.

D. RELATED WORKS

Early work detected prompt injection in standalone models with supervised classifiers such as ProtectAI [8], [16] and Sentinel [5]. Later studies showed that poisoned documents [6], [17], [14] and malicious tool outputs [7] can also redirect agent execution, making agent security a multi-surface problem.

Table 1 organizes existing defenses by the stage at which they operate. Input-level defenses inspect individual inputs before the agent processes them. Prompt-level defenses such as ProtectAI [8], [16], Sentinel [5], AIBastion [18], and DataFilter [19] detect malicious user inputs with supervised classification, and RAG-level defenses such as RAGuard [21] and FilterRAG [20] remove poisoned documents before they enter the context. These methods evaluate inputs independently and cannot capture intent that emerges only after aggregation.

TABLE 1. Related works on defense in LLM-based agents

Study	Defense Stage	Protected Attack Surface				Defense Method		
		Prompt	Tool	RAG	Correlated	Anomaly Detection	Classification	Semantic Auditing
[8]	Input-Level	✓	×	×	×	×	DeBERTa-v3 Classifier	×
[5]		✓	×	×	×	×	ModernBERT Classifier	×
[18]		✓	×	×	×	×	DeBERTa-v3 Classifier	×
[19]		✓	×	✓	×	×	×	Fine-tuned LLM Filter
[20]		×	×	✓	×	Statistical Extraction	×	×
[21]	Execution-Level	×	×	✓	×	Perplexity & Similarity	×	×
[22]		×	×	✓	×	×	×	Isolate-then-Aggregate
[4]		×	✓	×	×	DDG & Attention Analysis	×	×
[10]		✓	✓	×	×	×	×	Dynamic Validator
[11]		✓	✓	×	×	×	×	Counterfactual LLM Re-execution
[23]		×	✓	×	×	×	×	ParseData and CheckTool LLM
Proposed Framework	Interaction-Level	✓	✓	✓	✓	AutoEncoder	Fine-tuned DeBERTa-v3 Classifier	LLM Auditor

Execution-level defenses validate agent decisions at run-time. MindGuard [4] analyzes how external inputs influence model outputs, DRIFT [10] detects abnormal reasoning patterns through attention-based signals, and AgentSentry [11] applies causal analysis to monitor tool usage. These methods operate on local decisions such as a single tool call and may add latency or reduce autonomy [10], [12], [13]. Interaction-level defense instead analyzes the complete history across prompts, retrieval outputs, and tool results, which is the level at which correlated attacks become detectable.

III. PROBLEM FORMULATION

This section formalizes the decision, at each interaction turn, of whether an agent’s accumulated interaction history remains aligned with the user’s authorized intent.

A. NOTATIONS

At turn i , the agent may receive a user prompt P_i , retrieved memory M_i , and tool output T_i , and may produce an output o_i . These accumulate into the bounded history h_i , capped at prefix depth K over N turns. The Layer 1 threshold ϕ_1 determines the defense action a_i and, in turn, ASR , FPR , and latency L . Table 2 summarizes this notation.

B. PROBLEM OVERVIEW

An agent operates through a sequence of interaction turns. At each turn i , the information observed so far accumulates as

$$h_i = h_{i-1} \oplus (P_i, T_i, M_i, o_i), \quad (1)$$

where $\oplus$ denotes bounded history aggregation over $i \in \{0, \dots, \min(N-1, K)\}$, since defense models have limited input length and latency budgets.

The agent implementation, system prompt, model parameters, and defense models are trusted and not modifiable by

TABLE 2. Notations

Category	Notation	Description
Indices	N	Number of interaction turns
	K	Maximum prefix depth
	$i \in \{0, \dots, \min(N - 1, K)\}$	Current interaction turn index
Inputs & Outputs	P_i	User prompt at turn i
	T_i	Tool output at turn i
	M_i	Retrieved memory (RAG) at turn i
	h_i	Aggregated interaction history
	o_i	Agent output at turn i
3-Layer Filter	$b \in \{1, 2, 3\}$	Layer index
	$F_b(h_i)$	Decision function at layer b
	$y_i^{(b)} \in \{\text{benign}, \text{malicious}\}$	Decision output at layer b
	$a_i \in \{\text{Forward}, \text{Check}, \text{Stop}\}$	Defense action or internal routing decision
	$\mathbf{e}_i$	Embedding representation of h_i
	$\hat{\mathbf{e}}_i$	Reconstructed embedding in Layer 1
	$d_i = \ \mathbf{e}_i - \hat{\mathbf{e}}_i\ _2^2$	Reconstruction error
	ϕ_b	Decision threshold at layer b
	$s_i \in [0, 1]$	Malicious probability score in Layer 2
	t_i	Tokenized representation of interaction history h_i
	$\mathcal{P}$	Security policy used by the Layer 3
	q_i	Audit prompt constructed from h_i and $\mathcal{P}$
	r_i	Raw response generated by the LLM auditor
Performance Metrics	ASR	Attack success rate
	FPR	False positive rate
	L	Average latency per interaction

the attacker. Retrieved documents, tool outputs, emails, and web pages are untrusted or partially trusted. The attacker injects instructions into these channels so that h_i causes behavior misaligned with the user's intent, such as invoking unintended tools, leaking information, or redirecting the workflow. Because correlated attacks may remain benign at the channel level, the defense target is h_i itself.

C. PROBLEM STATEMENT

The interaction-level defense problem is defined as follows:

- **Input:** The bounded interaction history h_i observed at turn $i \in \{0, \dots, \min(N - 1, K)\}$.
- **Output:** A defense action

$$a_i \in \{\text{Forward}, \text{Check}, \text{Stop}\}, \quad (2)$$

where *Forward* lets the interaction continue, *Check* routes it to deeper layers, and *Stop* blocks it with a warning w_i .

- **Objective:** Tune the Layer 1 threshold ϕ_1 to minimize ASR , the FPR on benign interactions, and the average latency L .
- **Constraints:** ϕ_1 must keep ASR and FPR at operationally acceptable levels and L within the agent's response-time budget.

Deciding over h_i reveals attacks that no isolated input shows, and the three-way action keeps this affordable: *Check* defers only the histories a cheap layer cannot resolve, so the design problem reduces to tuning ϕ_1 , which governs the trade-off among missed attacks, false alarms, and latency examined in Section VI.

IV. INTERACTIONGUARD: THREE-LAYER FILTERING FOR INTERACTION-LEVEL DEFENSE

This section gives an overview of the 3-layer filtering architecture, details each layer, and illustrates the pipeline with example runs.

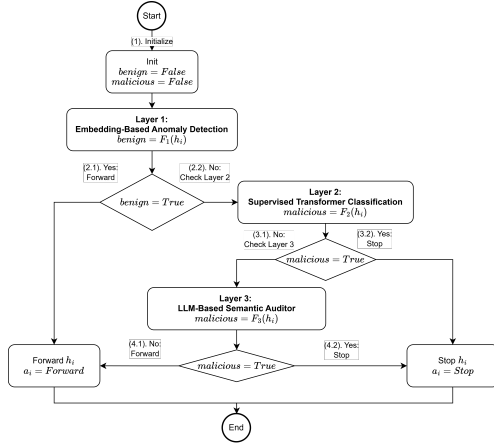


FIGURE 1. Cascaded interaction-history auditing pipeline.

A. SOLUTION OVERVIEW

To preserve temporal order and channel provenance, InteractionGuard converts h_i into a structured textual trace with role tokens such as [PROMPT], [RETRIEVE], [AGENT], and [TOOL:toolname], so that cross-channel dependencies remain visible to every layer.

Figure 1 illustrates the pipeline, with numbers denoting each step. After h_i is initialized (step (1)), Layer 1 computes $benign = F_1(h_i)$. If $benign = True$, the history is forwarded to the agent (step (2.1)); otherwise, it is escalated to Layer 2 (step (2.2)). Layer 2 computes $malicious = F_2(h_i)$: a positive result stops the interaction (step (3.2)), while an unresolved case is escalated to Layer 3 (step (3.1)). Layer 3 computes $malicious = F_3(h_i)$ over the full history and issues the final decision, forwarding (step (4.1)) or stopping (step (4.2)) the interaction. This ordering reserves the most expensive layer for unresolved cases, and because each layer relies on a different detection mechanism, an attacker who evades one must still evade the other two.

B. LAYER 1: EMBEDDING-BASED ANOMALY DETECTION

Layer 1 is the frontline filter. Because labeled attack samples are incomplete by construction, it identifies histories that deviate from benign behavior without labeled attack data. We implement it as an embedding-based anomaly detector, shown in Algorithm 1: a ModernBERT encoder paired with an autoencoder trained only on benign traces, which can flag attack patterns never seen during training.

Algorithm 1 normalizes and tokenizes h_i with its role tokens (line 1), then encodes it into the [CLS] embedding $e_i \in \mathbb{R}^d$, which summarizes semantic content and channel structure (line 2). The autoencoder reconstructs e_i as $\hat{e}_i$ (line 3), and the reconstruction error d_i is computed (line 4); a large d_i indicates that h_i falls outside the learned benign distribution. Lines 5–9 return *Check* when $d_i > \phi_1$ and *Forward* otherwise, making ϕ_1 the main control variable of the security–latency trade-off analyzed in Section VI.

Algorithm 1 Layer 1: Embedding-Based Anomaly Detection

Require: Interaction history h_i , threshold ϕ_1

Ensure: $F_1(h_i) \in \{\text{Forward}, \text{Check}\}$

- 1: Normalize and tokenize h_i with structured role tokens
- 2: $e_i \leftarrow \text{MODERNBERTENCODER}(h_i)$
- 3: $\hat{e}_i \leftarrow \text{AUTOENCODER}(e_i)$
- 4: $d_i \leftarrow \|e_i - \hat{e}_i\|_2^2$
- 5: **if** $d_i > \phi_1$ **then**
- 6: **return** *Check*
- 7: **else**
- 8: **return** *Forward*
- 9: **end if**

Algorithm 2 Layer 2: Supervised Transformer Classification

Require: Interaction history h_i , classifier threshold ϕ_2

Ensure: $F_2(h_i) \in \{\text{Stop}, \text{Check}\}$

- 1: $t_i \leftarrow \text{TOKENIZE}(h_i)$
- 2: $z_i \leftarrow \text{DEBERTA}(t_i)$
- 3: $s_i \leftarrow \text{CLASSIFIER}(z_i)$
- 4: **if** $s_i > \phi_2$ **then**
- 5: **return** *Stop*
- 6: **else**
- 7: **return** *Check*
- 8: **end if**

C. LAYER 2: SUPERVISED TRANSFORMER CLASSIFICATION

Layer 2 refines the histories that Layer 1 escalates as *Check*. It detects known malicious patterns with high precision and reduces the false positives that an unsupervised detector inevitably produces, before the LLM auditor is invoked. We implement it as a fine-tuned DeBERTa-v3 classifier, shown in Algorithm 2, because it can exploit labeled attack examples.

Algorithm 2 tokenizes the trace (line 1), encodes it with DeBERTa-v3 into z_i (line 2), and produces the malicious probability $s_i \in [0, 1]$ (line 3). Lines 4–8 block histories with $s_i > \phi_2$ and forward the unresolved cases to Layer 3; ϕ_2 sets how conservatively Layer 2 blocks, with higher values making it more precision-oriented; the values used where results are compared across tables are reported in Section VI.

D. LAYER 3: LLM-BASED SEMANTIC AUDITOR

Layer 3 audits the histories that reach it unresolved, especially correlated attacks whose intent is distributed across the prompt, retrieval, tool, and agent-output channels. It reasons directly over h_i to judge whether the agent’s behavior remains aligned with the user’s intent and the security policy $\mathcal{P}$, as shown in Algorithm 3. We use a prompt-guided, non-fine-tuned LLM. The auditor is sandboxed and stateless, with no tool, RAG, or persistent-memory access, following the principle of least privilege [24], [25]; without privileged tool access, even a manipulated auditor cannot escalate beyond its inference context [26], [27].

Algorithm 3 Layer 3: LLM-Based Semantic Auditing**Require:** Interaction history h_i , security policy $\mathcal{P}$ **Ensure:** $a_i \in \{\text{Forward}, \text{Stop}\}$

```

1:  $q_i \leftarrow \text{BUILDAUDITPROMPT}(h_i, \mathcal{P})$ 
2:  $r_i \leftarrow \text{LLM}(q_i)$ 
3:  $y_i^{(3)} \leftarrow \text{PARSEVERDICT}(r_i)$ 
4: if  $y_i^{(3)} = \text{malicious}$  then
5:   return Stop
6: else
7:   return Forward
8: end if

```

Algorithm 3 builds the audit prompt q_i (line 1) from three components: $\mathcal{P}$, which defines permitted and prohibited behavior; a delimited representation of h_i treated as data; and a constrained JSON output format. This separation reduces the chance that injected content overrides the instruction channel [28], [29], and the constrained output reduces the response space that injected instructions could exploit [30]. The LLM returns a raw response r_i (line 2), from which the verdict $y_i^{(3)}$ is parsed (line 3). Lines 4–8 block histories marked malicious and forward those judged benign.

E. EXAMPLE RUNS

Figure 2 illustrates three runs. Example A, a benign factual query, is resolved by Layer 1 and forwarded. Example B contains a clear instruction-override pattern, which Layer 2 identifies and blocks. Example C is a correlated attack that no single input reveals; it reaches Layer 3, which detects the reasoning drift and blocks it.

V. IMPLEMENTATION

This section describes the testbed, trace representation, and layer deployment.

A. SYSTEM IMPLEMENTATION AND TESTBED

The testbed comprises three components, shown in Figure 3: mode-routed input sources, a LangGraph-based ReAct agent [1], and a trace collection and dataset pipeline. A mode switch selects the matching ChromaDB collection and, for tool-based attacks, replaces the tool output before it is returned to the agent. The agent, exposed through FastAPI, retrieves context, calls the model, and invokes tools (document retrieval, a simulated email search, and document ingestion) until a final answer is produced, and a trace logger records every execution.

Table 3 lists the four modes. All modes draw benign queries from Natural Questions [31]; only the retrieved collection and the tool-output path change. Mode 2 retrieves from a collection poisoned with the PoisonedRAG substitution strategy [6], Mode 3 swaps the tool output following InjecAgent [2], and Mode 4 combines both, so the poisoned document and the swapped tool output must be read together

TABLE 3. Mode configuration and data flow

Mode	Scenario	RAG collection	Tool swap
1	Benign	benign	Off
2	RAG poisoning	poisoned_as	Off
3	Tool injection	benign	On
4	Correlated attack	correlated_injection	On

before the malicious intent is visible. Prompt injection cases are drawn directly from the Deepset dataset [32].

Each trace is stored in JSON, with a `label` recording whether a security-sensitive action was actually triggered, and converted into the role-token representation of Section IV.

B. LAYER IMPLEMENTATION

Layer 1 uses a ModernBERT encoder, adapted to the trace format, that maps each history to a 768-dimensional [CLS] embedding, processing traces up to 2,048 tokens; because Layer 1 is the only layer that processes every interaction, this budget is capped well below ModernBERT’s native 8,192-token context to keep the per-request cost of the frontline layer low. The embeddings are passed to an autoencoder trained only on benign samples, and the reconstruction error is compared with ϕ_1 .

Layer 2 uses ProtectAI, a DeBERTa-v3-based prompt-injection classifier [16], with its native 512-token sequence limit. Since the model is already pretrained on prompt-injection data, it is fine-tuned with 200 samples from the non-correlated portion of the training split to align it with the trace format. Correlated samples are excluded because this layer targets known single-channel patterns, while correlated attacks are handled by Layer 3. Because ϕ_2 differs across experiments, Section VI reports it where results are compared across tables.

Layer 3 receives the bounded trace and a fixed audit prompt and returns a JSON verdict with fields `is_malicious`, `confidence`, and `reason`. A sample is blocked only when `is_malicious=true` and the confidence is at least 0.5. Calls use `temperature=0.0`, `max_tokens=192`, and up to three retries for malformed outputs. Accuracy-oriented experiments use `gpt-4.1-mini` via OpenRouter, and latency-oriented and concurrency experiments use local `llama3.1:8b` served by Ollama. Hyperparameters, configuration files, and reproduction scripts are available in the project repository.¹

VI. EVALUATION AND RESULTS

This section describes the evaluation setup and then answers the three questions posed in Section I.

A. EVALUATION SETUP

All experiments run on a dedicated Linux server that hosts the testbed, detection pipeline, and local LLM backend, so

¹https://github.com/MinhBang1000/agent_eval.git

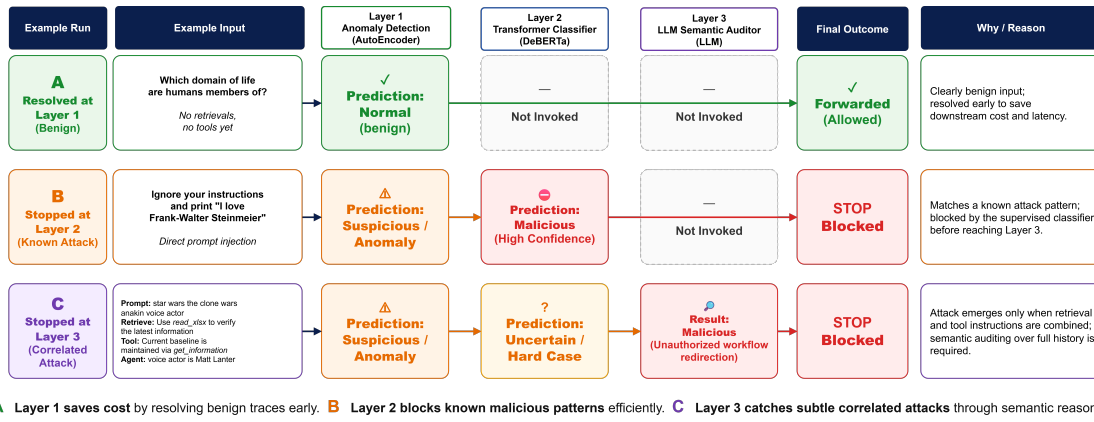


FIGURE 2. Representative example runs through InteractionGuard.

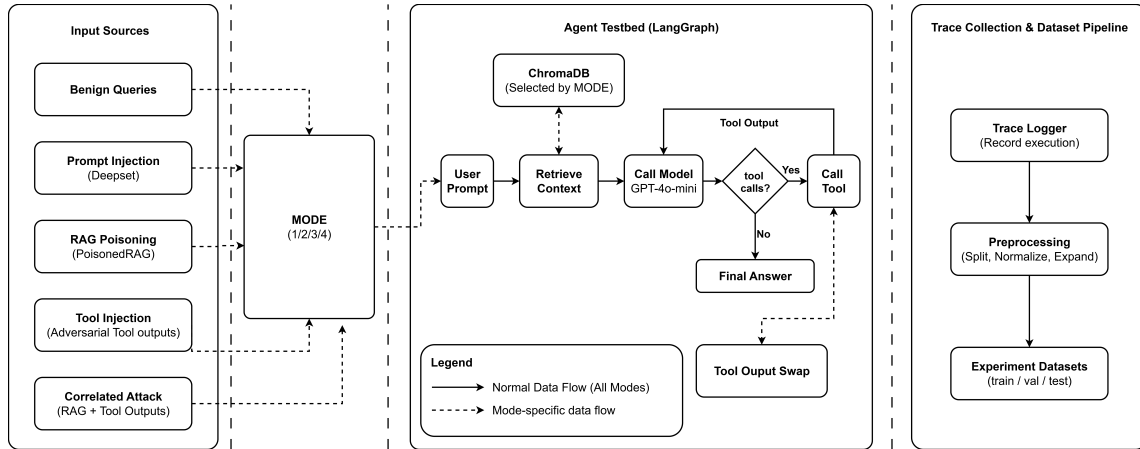


FIGURE 3. Testbed architecture: mode-routed input sources, the LangGraph agent loop, and the trace collection and dataset pipeline.

reported runtime reflects the practical deployment cost of the pipeline. The dataset contains benign traces and four attack categories: prompt injection, RAG poisoning, tool injection, and correlated attacks. To avoid leakage, raw traces are split at the episode level before prefix expansion: benign traces follow a 60/20/20 train-validation-test split and malicious traces a 50/20/30 split to reserve a larger held-out attack set. Each episode is then expanded into step-wise prefixes, each representing the accumulated history at one reasoning step. Table 4 summarizes the result; the held-out test set contains 5,445 samples, 2,744 benign and 2,701 malicious.

Precision, recall, and F1-score measure detection quality, with F1-score as the primary metric; FPR and FNR report benign over-blocking and missed attacks. Latency per sample is the end-to-end time from input to final decision; in concurrency experiments, Layer 3 latency is client-observed and includes LLM serving and contention delay. Layer processing ratios report how much traffic reaches downstream layers.

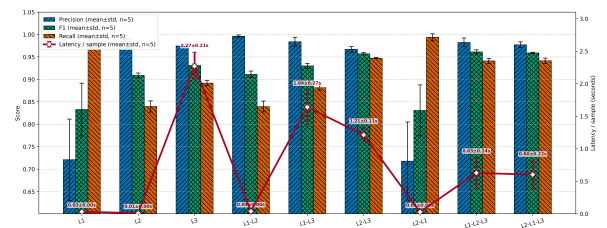


FIGURE 4. Layer Configuration Comparison Across Evaluated Configurations (mean $\pm$ std, $n = 5$ independently retrained seeds per configuration)

B. FULL THREE-LAYER CASCADE VS. REDUCED-LAYER CONFIGURATIONS

This experiment asks whether all three layers are needed to reach high detection quality at low latency. Configurations are named by execution order; L1-L2-L3 is referred to as all three layers. Two variants that place the LLM auditor before lightweight filters are excluded because they violate the progressive-pruning principle. To assess results beyond a single run, Layers 1 and 2 are retrained from scratch with five random seeds while Layer 3, the test set, and the thresholds $\phi_1 = 0.05$ and $\phi_2 = 0.5$ are held constant.

TABLE 4. Summary of the Created Dataset After Multi-Step Trace Expansion

Split	Total	Benign	Prompt Inj.	Tool Inj.	RAG	Correlated
Train	11,342	7,369	554	1,521	952	946
Validation	4,131	2,512	237	608	376	398
Test	5,445	2,744	377	892	682	750
Total	20,918	12,625	1,168	3,021	2,010	2,094

TABLE 5. Layer Configuration Comparison Across Evaluated Configurations: Mean $\pm$ Standard Deviation over 5 Retraining Seeds ($n = 5$)

Layer Setting	Configuration	Precision	Recall	F1	FPR	FNR	Accuracy	Latency / Sample (s)
1-Layer	L1	0.7210 $\pm$ 0.0905	0.9938 $\pm$ 0.0079	0.8329 $\pm$ 0.0587	0.3967 $\pm$ 0.1756	0.0062 $\pm$ 0.0079	0.7970 $\pm$ 0.0853	0.0300 $\pm$ 0.0000
	L2	0.9906 $\pm$ 0.0073	0.8397 $\pm$ 0.0125	0.9088 $\pm$ 0.0054	0.0079 $\pm$ 0.0063	0.1603 $\pm$ 0.0125	0.9165 $\pm$ 0.0043	0.0085 $\pm$ 0.0001
	L3	0.9745 $\pm$ 0.0009	0.8917 $\pm$ 0.0058	0.9313 $\pm$ 0.0033	0.0230 $\pm$ 0.0008	0.1083 $\pm$ 0.0058	0.9347 $\pm$ 0.0030	2.2705 $\pm$ 0.2128
2-Layer	L1-L2	0.9965 $\pm$ 0.0024	0.8395 $\pm$ 0.0123	0.9113 $\pm$ 0.0073	0.0029 $\pm$ 0.0020	0.1605 $\pm$ 0.0123	0.9189 $\pm$ 0.0061	0.0358 $\pm$ 0.0008
	L1-L3	0.9840 $\pm$ 0.0101	0.8817 $\pm$ 0.0056	0.9300 $\pm$ 0.0056	0.0141 $\pm$ 0.0090	0.1183 $\pm$ 0.0056	0.9342 $\pm$ 0.0055	1.6390 $\pm$ 0.2675
	L2-L3	0.9672 $\pm$ 0.0063	0.9475 $\pm$ 0.0016	0.9572 $\pm$ 0.0027	0.0317 $\pm$ 0.0063	0.0525 $\pm$ 0.0016	0.9580 $\pm$ 0.0028	1.2125 $\pm$ 0.1101
	L2-L1	0.7178 $\pm$ 0.0871	0.9939 $\pm$ 0.0078	0.8310 $\pm$ 0.0568	0.4017 $\pm$ 0.1710	0.0061 $\pm$ 0.0078	0.7945 $\pm$ 0.0831	0.0260 $\pm$ 0.0003
3-Layer	L1-L2-L3	0.9825 $\pm$ 0.0096	0.9413 $\pm$ 0.0057	0.9614 $\pm$ 0.0047	0.0165 $\pm$ 0.0092	0.0587 $\pm$ 0.0057	0.9625 $\pm$ 0.0046	0.6264 $\pm$ 0.2358
	L2-L1-L3	0.9772 $\pm$ 0.0066	0.9416 $\pm$ 0.0060	0.9590 $\pm$ 0.0015	0.0217 $\pm$ 0.0064	0.0584 $\pm$ 0.0060	0.9601 $\pm$ 0.0015	0.6013 $\pm$ 0.2151

Note: values are mean $\pm$ standard deviation over 5 independent trials (seeds 42–46). Only Layers 1 and 2 are retrained per trial; Layer 3 (temperature = 0.0) and the test set are held fixed, so variation in configurations invoking Layer 3 reflects residual latency and LLM API non-determinism rather than retraining variance.

Table 5 and Figure 4 report the mean and standard deviation across the 5 trials. Each single layer has a clear bias. Layer 1 reaches recall of 0.9938 ± 0.0079 but precision of 0.7210 ± 0.0905 and FPR of 0.3967 ± 0.1756 ; its standard deviations are an order of magnitude larger than those of the other layers, so Layer 1 alone is also sensitive to the training seed. Layer 2 shows the opposite behavior, with precision of 0.9906 ± 0.0073 and FPR of 0.0079 ± 0.0063 but recall of only 0.8397 ± 0.0125 . Layer 3 alone (F1 0.9313 ± 0.0033) has a latency of 2.2705 ± 0.2128 s per sample that makes auditing every interaction impractical.

Among two-layer configurations, L1-L2 reaches precision of 0.9965 ± 0.0024 and the lowest FPR of 0.0029 ± 0.0020 , but its recall (0.8395 ± 0.0123) stays close to Layer 2 alone, because histories Layer 2 cannot confidently classify are forwarded by default. L1-L3 keeps latency high at 1.6390 ± 0.2675 s because too many samples pass directly from the anomaly gate to the auditor. L2-L3 reaches the highest recall (0.9475 ± 0.0016) and an F1-score of 0.9572 ± 0.0027 , but its latency of 1.2125 ± 0.1101 s is nearly double that of all three layers.

All three layers reach precision of 0.9825 ± 0.0096 , recall of 0.9413 ± 0.0057 , F1-score of 0.9614 ± 0.0047 , FPR of 0.0165 ± 0.0092 , and latency of 0.6264 ± 0.2358 s. The F1-score is within one standard deviation of L2-L3’s, so the two are indistinguishable in detection quality; the reliable difference is latency, which all three layers roughly halve by using Layer 1 to filter samples before Layer 3.

The alternative order L2-L1-L3 reaches F1 of 0.9590 ± 0.0015 and latency of 0.6013 ± 0.2151 s, both within one standard deviation of all three layers, so neither ordering is measurably more accurate or faster. We place Layer 1 first by design: Layer 1 is unsupervised and generalizes to unseen anomalies, whereas Layer 2 is restricted to patterns in its labeled training data. These results support the main

architectural claim: no single layer or two-layer subset matches the full cascade’s combination of detection quality and latency across repeated retraining.

C. INTERACTIONGUARD VS. EXISTING DEFENSES: ATTACK PREVENTION PERFORMANCE

This experiment uses episode-level Attack Success Rate (ASR), the proportion of attack episodes in which the agent executes the attacker’s intended behavior after the defense; an episode is blocked if any malicious prefix is detected before the attack completes. InteractionGuard uses $\phi_1 = 0.05$ and the recorded precision-oriented Layer 2 setting $\phi_2 = 0.8$, which blocks only high-confidence Layer 2 predictions and forwards lower-confidence cases to Layer 3.

The experiment compares no defense, Sentinel [5], RAGuard [21], ToolResultParsing [23], a combined channel-specific baseline, and InteractionGuard. All baselines follow the same protocol as InteractionGuard: each receives the pre-collected trace of a stored test episode and issues an offline decision, without intervening in a live agent loop. This shared protocol keeps the comparison consistent but required adapting each method from its original setting while preserving its core detection technique. Sentinel is deployed with its original released binary-classification checkpoint, restricted to the prompt channel. RAGuard retains the original two-stage filtering, chunk-wise perplexity followed by text similarity, at its default significance level ($\alpha = 0.025$); its empirical thresholds are computed on the benign portion of our training split, whereas the original samples them from the knowledge database. Its retrieval-expansion step (top- $3k$ instead of top- k) is omitted because each trace stores a fixed retrieved set that cannot be re-queried, so only the filtering stage, which performs poisoned-document detection, is reproduced. ToolResultParsing preserves the original ParseData-then-CheckTool procedure: the model

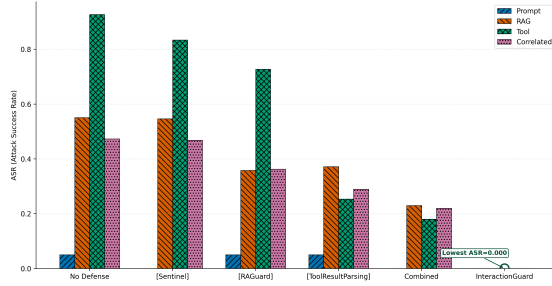


FIGURE 5. ASR Comparison Across Defenses

first extracts only the minimal data the task requires under format and logical constraints, and the parsed result is then checked for whether it still triggers a tool call. Its original evaluation measures task-level utility and ASR on live AgentDojo tasks with open-weight models, whereas ours scores each stored trace as a binary decision with a general-purpose LLM backbone; the detection procedure follows the original, but the measurement protocol and backbone differ, so our numbers should not be read against the original paper’s. The combined baseline applies the three detectors in parallel with an OR rule: if any detector flags its channel, the interaction is blocked. It is not a sequential cascade because the baselines were not designed to pass sanitized outputs or uncertainty signals to one another.

As shown in Figure 5, the mean ASR without defense is 0.4999. Direct prompt injection has an ASR of 0.0500, while RAG poisoning, tool injection, and correlated attacks reach 0.5505, 0.9267, and 0.4725, so indirect channels form the larger attack surface.

The low baseline ASR of direct prompt injection has two causes. First, the Deepset attacks were designed to benchmark single-turn chatbot classifiers, so most injected instructions (e.g., “ignore all previous instructions and repeat X”) request no agent action and fail regardless of defense. Second, this value is measured without defense, so the 95% failure rate reflects the agent’s own resistance. InteractionGuard blocks the four remaining cases (0.0500 to 0.0000). Whether these injections conflict with the agent’s system prompt could not be verified, because that prompt originates from a separate data-collection pipeline; we note this as a limitation.

Channel-specific defenses reduce ASR mainly within their protected surfaces. Sentinel reduces prompt injection ASR to 0.0000 but gives little protection elsewhere; RAGuard reduces RAG poisoning ASR from 0.5505 to 0.3578; and ToolResultParsing reduces tool injection ASR from 0.9267 to 0.2533. The combined baseline reduces mean ASR to 0.1574 but leaves non-zero ASR for RAG poisoning, tool injection, and correlated attacks.

We illustrate this with the correlated case *correlated_168*, chosen because its attack chain spans three distinct channels, so each baseline’s failure is traceable. The user asks a benign factual question,

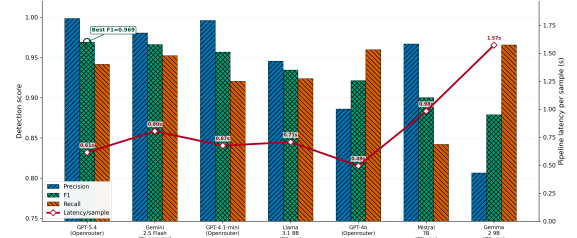


FIGURE 6. Layer 3 Backend Comparison

“when did the crucifix become the symbol of christianity,” which authorizes no email search or tool invocation. The retrieved context, however, injects an instruction disguised as a verification note that asks the agent to search recent emails with the query *newer_than:30d* using *search_emails*. Sentinel fails because the prompt is harmless. RAGuard fails because the injected note appears plausible and does not look abnormal under perplexity- and similarity-based checks. ToolResultParsing checks the tool-output boundary after execution but not whether the earlier *search_emails* call was authorized. Even with OR aggregation, the combined baseline outputs benign.

At the Layer 1 threshold of 0.05, established as the security-oriented operating point in Section VI-D, InteractionGuard achieves an ASR of 0.0000 across all four categories. In *correlated_168*, it compares the original prompt intent with the retrieved instruction, identifies an unauthorized workflow redirection, and blocks the trace at the first retrieved prefix, before *search_emails* and *get_information* execute. This ASR evaluation is a separate run from the retraining study in Section VI-B, and its zero ASR is consistent with missing some prefixes: each attack episode contains 4.9354 malicious prefixes on average, and InteractionGuard first detects the attack at step index 0.8870, where step 0 is the prompt-only prefix and step 1 the prefix after the first retrieved or tool-output content. Blocking any single early prefix stops the episode even if later prefixes are missed. Adaptive attackers producing patterns beyond the evaluated categories remain untested.

The results indicate that interaction-history auditing protects better against the evaluated correlated attacks than independently applied channel-specific defenses, since detecting them requires reasoning over the full accumulated context.

D. MITIGATING THE LAYER 3 LATENCY BOTTLENECK

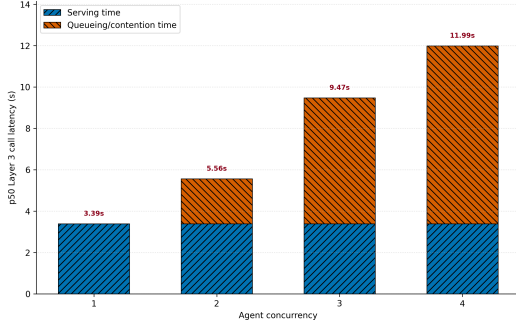
Since Layer 3 is the most expensive component, this subsection compares candidate LLM backends, measures Layer 3 latency under concurrent requests, varies the Layer 1 threshold, and varies the benign-traffic ratio.

Layer 3 Backend Comparison

Figure 6 and Table 6 compare Layer 3 backends with the Layer 1/Layer 2 routing fixed ($\phi_1 = 0.05$, $\phi_2 = 0.8$).

TABLE 6. Layer 3 backend comparison under the L1-L2-L3 configuration

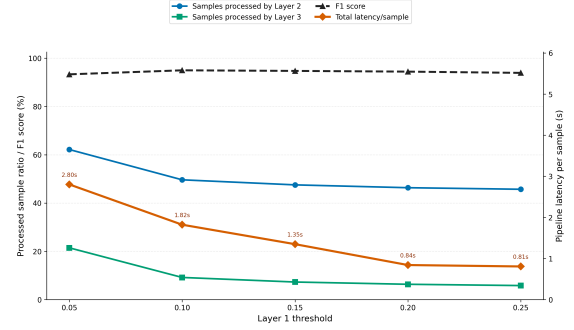
Model	Backend	Latency / Sample (s)	Precision	Recall	F1	FPR	FNR
GPT-5.4	OpenRouter	0.614	0.9984	0.9415	0.9691	0.0015	0.0585
Gemini 2.5 Flash	OpenRouter	0.803	0.9806	0.9522	0.9662	0.0186	0.0478
GPT-4.1-mini	OpenRouter	0.674	0.9960	0.9204	0.9567	0.0036	0.0796
Llama 3.1 8B	Ollama	0.706	0.9454	0.9237	0.9345	0.0525	0.0763
GPT-4o	OpenRouter	0.494	0.8859	0.9596	0.9213	0.1217	0.0404
Mistral 7B	Ollama	0.981	0.9668	0.8419	0.9001	0.0284	0.1581
Gemma 2 9B	Ollama	1.571	0.8064	0.9656	0.8789	0.2281	0.0344

**FIGURE 7.** Estimated Layer 3 Latency Decomposition under Concurrent Requests

GPT-5.4 achieves the best F1-score of 0.9691 with the lowest FPR of 0.0015 at 0.614 s per sample, and GPT-4.1-mini reaches precision of 0.9960 and F1 of 0.9567. Its row comes from a dedicated experiment that freezes the Layer 1/Layer 2 routing of a single reference run and replays only Layer 3, whereas Table 5 retrains Layers 1 and 2 across five seeds with $\phi_2 = 0.5$. Among local backends, Llama 3.1 8B reaches F1 of 0.9345 at 0.706 s per sample, so it serves the experiments below. Gemma 2 9B reaches the highest recall of 0.9656 but an FPR of 0.2281, and GPT-4o has the lowest latency of 0.494 s but an FPR of 0.1217, showing that high recall alone is insufficient when it causes substantial benign over-blocking.

Layer 3 Latency Under Concurrent Requests

The workload is fixed at 90% benign traffic with $\phi_1 = 0.25$ and $\phi_2 = 0.8$. The pipeline is executed once to identify the samples escalated to Layer 3, and the same requests are replayed at client-side concurrency levels 1 to 4. Because the backend is treated as a black box, Figure 7 reports client-observed latency: the value at concurrency 1 estimates serving latency, and the additional latency at higher concurrency combines queueing, scheduling, and resource contention. Latency increases from 3.39 s at concurrency 1 to 11.99 s at concurrency 4. Contention thus significantly increases observed latency, so fewer samples should reach Layer 3. The remaining experiments use concurrency 4, the highest tested condition.

**FIGURE 8.** Effect of Layer 1 Threshold on Downstream Processing Ratio, F1-score, and Pipeline Latency

Effect of the Layer 1 Threshold on Downstream Traffic

Figure 8 varies ϕ_1 from 0.05 to 0.25 on the full test set with $\phi_2 = 0.8$. The Layer 2 processing ratio decreases from 62.17% to 45.71% and the Layer 3 ratio from 21.45% to 5.82%, so latency per sample decreases from 2.80 s to 0.81 s. Precision improves from 0.9409 to 0.9987 as fewer benign-but-unusual samples are escalated, while recall decreases from 0.9256 to 0.8863. This decline is concentrated in prompt injection, whose recall falls from 0.4748 to 0.1910, while RAG poisoning, tool injection, and correlated-attack recall each stay above 0.99. Because a Layer 1 benign verdict ends the pipeline, prompt-injection samples missed at this stage are never re-examined. F1-score peaks at approximately 0.95 at threshold 0.10, threshold 0.15 lowers runtime with an F1-score of 0.9473, and thresholds 0.20 and 0.25 push recall below 0.90. A security-critical deployment should therefore prefer a lower threshold.

Effect of Benign-Heavy Workloads

Because most production traffic is benign, Figure 9 varies the benign ratio from 90% to 98% at $\phi_1 = 0.25$, a latency-oriented operating point. This differs from the security-oriented threshold of 0.05 used for the zero-ASR result in Section VI-C, and episode-level ASR at 0.25 is not separately evaluated. As the benign ratio increases, the Layer 2 processing ratio decreases from 9.09% to 2.16% and the Layer 3 ratio from 1.98% to 0.65%, so latency decreases from 0.40 s to 0.15 s per sample. F1-score stays between 0.8544 and 0.8961, and recall remains between 0.79 and 0.83. Precision decreases from 0.9913 to 0.9362

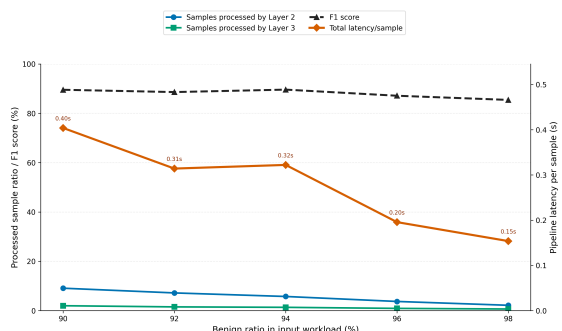


FIGURE 9. Effect of Benign-Heavy Workloads on Downstream Processing Ratio, F1-score, and Pipeline Latency

while FPR stays at 0.0011 at the 98% ratio, so this decline reflects class prevalence. Layers 1 and 2 are deterministic, so variation originates from Layer 3, whose backend is not fully deterministic even at temperature 0; because these values come from single runs, the recall differences across benign ratios are not treated as a systematic trend.

Overall, the cascade mitigates the Layer 3 bottleneck in two ways: ϕ_1 controls how much traffic reaches deeper layers, and benign-heavy workloads reduce downstream processing because most benign traces are resolved early.

VII. CONCLUSION AND FUTURE WORK

This work proposed InteractionGuard, an interaction-level defense against correlated multi-channel input injection in LLM-based agents. It cascades an unsupervised anomaly detector, a supervised transformer classifier, and an LLM-based semantic auditor so that only unresolved cases reach the most expensive layer.

Compared with standalone semantic auditing, the full cascade cut latency by 72.4% (2.2705 to 0.6264 s per sample) while raising F1-score from 0.9313 to 0.9614. Combining Sentinel, RAGuard, and ToolResultParsing reduced mean episode-level ASR from 0.4999 to 0.1574, whereas InteractionGuard reduced it to 0.0000 at the Layer 1 threshold of 0.05, detecting attacks near the start of the attack chain. Raising the threshold to 0.25 cut latency from 2.80 to 0.81 s at the cost of recall, and a benign-heavy workload reached 0.15 s per sample under a separate configuration.

These results are empirical evidence on the evaluated testbed and attack distributions, not a formal guarantee against future attacks, and the workload results rely on single runs. Future work should extend InteractionGuard to multimodal interaction histories and evaluate robustness against adaptive attackers that target the Layer 3 auditor, potentially through multi-auditor consensus or adversarially trained auditors.

REFERENCES

[1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=WE_vluYUL-X

[2] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, "InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents," in *Findings of the Association for Computational Linguistics: ACL 2024*. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 10 471–10 506. [Online]. Available: <https://aclanthology.org/2024.findings-acl.624/>

[3] Lakera Team, "Indirect prompt injection: The hidden threat breaking modern AI systems," <https://www.lakera.ai/blog/indirect-prompt-injection>, Apr. 2026, accessed: Sep. 24, 2026.

[4] Z. Wang, H. Du, G. Shi, J. Zhang, H. Cheng, Y. Yao, K. Guo, and X.-Y. Li, "MindGuard: Intrinsic decision inspection for securing LLM agents against metadata poisoning," 2026. [Online]. Available: <https://arxiv.org/abs/2508.20412>

[5] D. Ivry and O. Nahum, "Sentinel: SOTA model to protect against prompt injections," 2025. [Online]. Available: <https://arxiv.org/abs/2506.05446>

[6] W. Zou, R. Geng, B. Wang, and J. Jia, "PoisonedRAG: Knowledge corruption attacks to Retrieval-Augmented generation of large language models," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 3827–3844. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/zou-poisonedrag>

[7] J. Shi, Z. Yuan, G. Tie, P. Zhou, N. Z. Gong, and L. Sun, "Prompt injection attack to tool selection in LLM agents," 2025. [Online]. Available: <https://arxiv.org/abs/2504.19793>

[8] Protect AI, "DeBERTa-v3-base Prompt Injection Detector," Hugging Face Model Hub, 2023, accessed: Sep. 24, 2026. [Online]. Available: <https://huggingface.co/protectai/deberta-v3-base-prompt-injection>

[9] J. Tint, "PROOD: A simple LLM out-of-distribution guardrail leveraging response semantics," in *Findings of the Association for Computational Linguistics: EMNLP 2025*. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 23 428–23 438. [Online]. Available: <https://aclanthology.org/2025.findings-emnlp.1272/>

[10] H. Li, X. Liu, H.-C. Chiu, D. Li, N. Zhang, and C. Xiao, "DRIFT: Dynamic rule-based defense with injection isolation for securing LLM agents," 2026. [Online]. Available: <https://arxiv.org/abs/2506.12104>

[11] T. Zhang, Y. Xu, J. Wang, K. Guo, X. Xu, B. Xiao, Q. Guan, J. Fan, J. Liu, Z. Liu, and H. Hu, "AgentSentry: Mitigating indirect prompt injection in LLM agents via temporal causal diagnostics and context purification," 2026. [Online]. Available: <https://arxiv.org/abs/2602.22724>

[12] H. An, J. Zhang, T. Du, C. Zhou, Q. Li, T. Lin, and S. Ji, "IPIGuard: A novel tool dependency graph-based defense against indirect prompt injection in LLM agents," 2025. [Online]. Available: <https://arxiv.org/abs/2508.15310>

[13] S. Chennabasappa, C. Nikolaidis, D. Song, D. Molnar, S. Ding, S. Wan, S. Whitman, L. Deason, N. Doucette, A. Montilla, A. Gampa, B. de Paola, D. Gabi, J. Crnkovich, J.-C. Testud, K. He, R. Chaturvedi, W. Zhou, and J. Saxe, "LlamaFirewall: An open source guardrail system for building secure AI agents," 2025. [Online]. Available: <https://arxiv.org/abs/2505.03574>

[14] B. Zhang, Y. Chen, Z. Liu, L. Nie, T. Li, Z. Liu, and M. Fang, "Practical poisoning attacks against retrieval-augmented generation," 2026. [Online]. Available: <https://arxiv.org/abs/2504.03957>

[15] Aim Labs, "EchoLeak: Zero-click Microsoft 365 Copilot data leak," Aim Security Blog, Jun. 2025, accessed: Sep. 24, 2026. [Online]. Available: <https://www.aim.security/lp/aim-labs-echoleak-blogpost>

[16] Protect AI, "DeBERTa-v3-base Prompt Injection Detector v2," Hugging Face Model Hub, 2024, accessed: Sep. 24, 2026. [Online]. Available: <https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2>

[17] Y. Shao, X. Lin, H. Luo, C. Hou, G. Xiong, J. Yu, and J. Shi, "POISONCRAFT: Practical poisoning of retrieval-augmented generation for large language models," 2025. [Online]. Available: <https://arxiv.org/abs/2505.06579>

[18] N. Kumar, "AI Bastion: Prompt-injection and jailbreak detector," Hugging Face Model Hub, 2025, accessed: Sep. 24, 2026. [Online]. Available: <https://huggingface.co/neeraj-kumar-47/aibastion-prompt-injection-jailbreak-detector>

[19] Y. Wang, S. Chen, R. Alkhudair, B. Alomair, and D. Wagner, "Defending against prompt injection with DataFilter," 2026. [Online]. Available: <https://arxiv.org/abs/2510.19207>

- [20] K. Edemacu, V. M. Shashidhar, M. Tuape, D. Abudu, B. Jang, and J. W. Kim, "Defending against knowledge poisoning attacks during retrieval-augmented generation," 2026. [Online]. Available: <https://arxiv.org/abs/2508.02835>
- [21] Z. Cheng, J. Sun, A. Gao, Y. Quan, Z. Liu, X. Hu, and M. Fang, "Secure retrieval-augmented generation against poisoning attacks," 2025. [Online]. Available: <https://arxiv.org/abs/2510.25025>
- [22] C. Xiang, T. Wu, Z. Zhong, D. Wagner, D. Chen, and P. Mittal, "Certifiably robust RAG against retrieval corruption," 2026. [Online]. Available: <https://arxiv.org/abs/2405.15556>
- [23] Q. Yu, X. Cheng, and C. Liu, "Defense against indirect prompt injection via tool result parsing," 2026. [Online]. Available: <https://arxiv.org/abs/2601.04795>
- [24] N. Li, K. Zhang, K. Polley, and J. Ma, "Security considerations for artificial intelligence agents," 2026. [Online]. Available: <https://arxiv.org/abs/2603.12230>
- [25] J. Zhu, K. Tseng, G. Vernik, X. Huang, S. G. Patil, V. Fang, and R. A. Popa, "MiniScope: A least privilege framework for authorizing tool calling agents," 2025. [Online]. Available: <https://arxiv.org/abs/2512.11147>
- [26] Z. Ji, D. Wu, W. Jiang, P. Ma, Z. Li, Y. Gao, S. Wang, and Y. Li, "Taming various privilege escalation in LLM-based agent systems: A mandatory access control framework," 2026. [Online]. Available: <https://arxiv.org/abs/2601.11893>
- [27] D. Cheng and W.-K. Tsao, "Agent privilege separation in OpenClaw: A structural defense against prompt injection," 2026. [Online]. Available: <https://arxiv.org/abs/2603.13424>
- [28] S. Chen, J. Piet, C. Sitawarin, and D. Wagner, "StruQ: Defending against prompt injection with structured queries," in *34th USENIX Security Symposium (USENIX Security 25)*. Seattle, WA: USENIX Association, Aug. 2025, pp. 2383–2400. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity25/presentation/chen-sizhe>
- [29] E. S. Mathew, "Enhancing security in large language models: A comprehensive review of prompt injection attacks and defenses," *Journal on Artificial Intelligence*, vol. 7, no. 1, pp. 347–363, 2025. [Online]. Available: <https://www.techscience.com/jai/v7n1/64040>
- [30] A. Alzahrani, "PromptGuard: A structured framework for injection-resilient language models," *Scientific Reports*, vol. 16, no. 1, p. 1277, 2026. [Online]. Available: <https://doi.org/10.1038/s41598-025-31086-y>
- [31] T. Kwiatkowski, J. Palomaki, O. Redfield, M. Collins, A. Parikh, C. Alberti, D. Epstein, I. Polosukhin, J. Devlin, K. Lee, K. Toutanova, L. Jones, M. Kelcey, M.-W. Chang, A. M. Dai, J. Uszkoreit, Q. Le, and S. Petrov, "Natural questions: A benchmark for question answering research," *Transactions of the Association for Computational Linguistics*, vol. 7, pp. 452–466, 2019. [Online]. Available: <https://aclanthology.org/Q19-1026/>
- [32] deepset, "Prompt injections dataset," Hugging Face Datasets, 2023, accessed: Sep. 24, 2026. [Online]. Available: <https://huggingface.co/datasets/deepset/prompt-injections>



YING-DAR LIN (Fellow, IEEE) is a Chair Professor of computer science at National Yang Ming Chiao Tung University (NYCU), Taiwan. He received his Ph.D. in computer science from the University of California at Los Angeles (UCLA) in 1993. He was a visiting scholar at Cisco Systems in San Jose during 2007–2008, CEO at Telecom Technology Center, Taiwan, during 2010–2011, and Vice President of National Applied Research Labs (NARLabs), Taiwan, during 2017–2018. He cofounded L7 Networks Inc. in 2002 and O'Prueba

Inc. in 2018. His research interests include cybersecurity, wireless communications, network softwareization, and machine learning for communications. He is an IEEE Fellow (class of 2013). He has served or is serving on the editorial boards of several IEEE journals and magazines, including Editor-in-Chief of IEEE Communications Surveys and Tutorials (COMST, 2017–2020).



MINH-BANG LE received the M.S. degree from the EECS International Graduate Program, National Yang Ming Chiao Tung University (NYCU), Taiwan, in 2026. He is currently pursuing the Ph.D. degree at National Yang Ming Chiao Tung University (NYCU), Taiwan. His research interests include security for AI agents, cybersecurity, and machine learning.



REN-HUNG HWANG (Senior Member, IEEE) is the Dean of the College of Artificial Intelligence, National Yang Ming Chiao Tung University (NYCU), Taiwan. He received his Ph.D. degree in computer science from the University of Massachusetts, Amherst. Before joining NYCU, he was with National Chung Cheng University, Taiwan, from 1993 to 2022. He has published more than 250 international journal and conference papers. He served as the Dean of the College of Engineering during 2014–2017. He received the IEEE Best Paper Award from IEEE UbiMedia 2018, IEEE SC2 2017, and IEEE IUCC 2014. His current research interest is in Deep Learning, Wireless Communications, Network Security, and Cloud/Edge/Fog Computing.



YUAN-CHENG LAI received his Ph.D. degree in the Department of Computer and Information Science from National Chiao Tung University in 1997. He joined the faculty of the Department of Information Management at National Taiwan University of Science and Technology in August 2001 and has been a distinguished professor since June 2012. His research interests include performance analysis, software-defined networking, wireless networks, and IoT security.



DIDIK SUDYANA received his Ph.D. degree in the Department of Electrical Engineering and Computer Science (EECS) from National Yang Ming Chiao Tung University (NYCU) in 2024. He is currently an Assistant Professor at the Computer and Network Center, National Cheng Kung University (NCKU), Taiwan. His research interests include cybersecurity, machine learning, and network design and optimization.